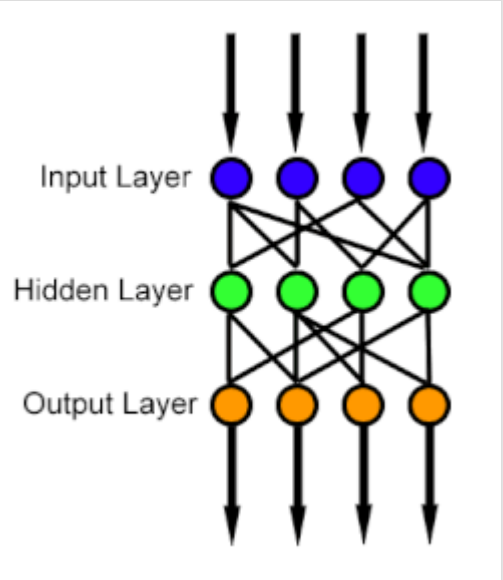
Feedforward neural network

A **feedforward neural network** is an artificial neural network in which information flows in a single direction – inputs are multiplied by weights to obtain outputs (inputs-to-output).[2] It contrasts with a recurrent neural network, in which loops allow information from later processing stages to feed back to earlier stages.[3] Feedforward multiplication is essential for backpropagation,[4][5][6][7][8] because feedback, where the outputs feed back to the very same inputs and modify them, forms an infinite loop which is not possible to differentiate through backpropagation. This nomenclature appears to be a point of confusion between some computer scientists and scientists in other fields studying brain networks.[9]



In a feedforward network, information always moves in one direction; it never goes backwards.

Mathematical foundations

Activation function

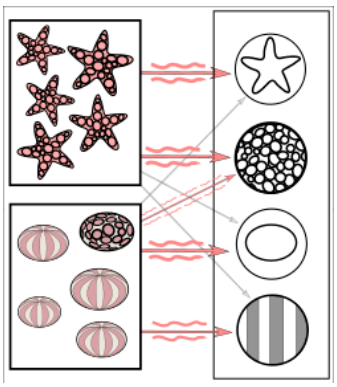
The two historically common activation functions are both sigmoids, and are described by

$$y(v_i) = \tanh(v_i) \text{ and } y(v_i) = (1 + e^{-v_i})^{-1}.$$

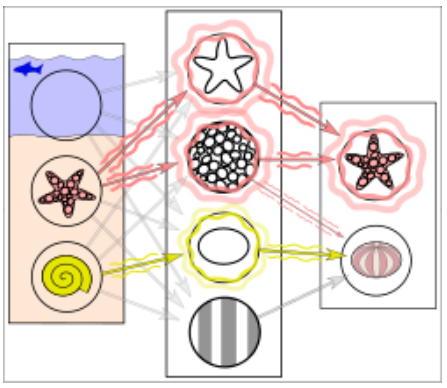
The first is a hyperbolic tangent that ranges from -1 to 1, while the other is the logistic function, which is similar in shape but ranges from 0 to 1. Here y_i is the output of the i-th node (neuron) and v_i is the weighted sum of the input connections. Alternative activation functions have been proposed, including the rectifier and softplus functions. More specialized activation functions include radial basis functions (used in radial basis networks, another class of supervised neural network models).

In recent developments of deep learning, the rectified linear unit (ReLU) is more frequently used as one of the possible ways to overcome the numerical problems related to the sigmoids.

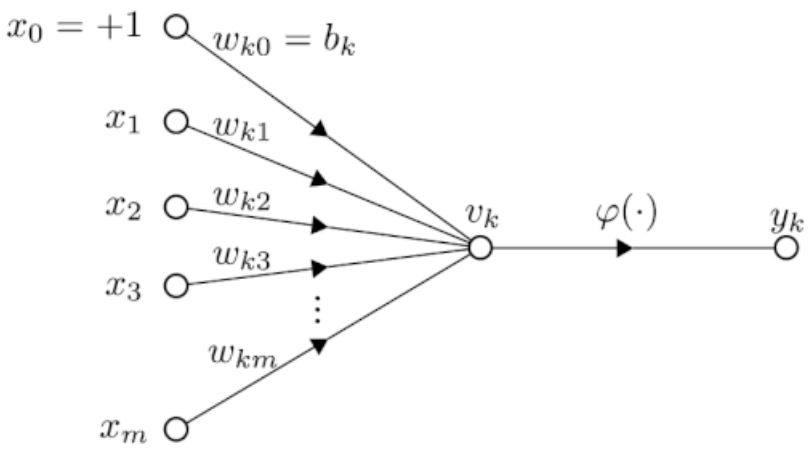
Learning



Simplified example of training a neural network for object detection: The network is trained on multiple images depicting either starfish or sea urchins, which are correlated with "nodes" that represent visual features. The starfish match with a ringed texture and a star outline, whereas most sea urchins match with a striped texture and oval shape. However, the instance of a ring-textured sea urchin creates a weakly weighted association between them.



Subsequent run of the network on an input image (left):[1] The network correctly detects the starfish. However, the weakly weighted association between ringed texture and sea urchin also confers a weak signal to the latter from one of two intermediate nodes. In addition, a shell that was not included in the training gives a weak signal for the oval shape, also resulting in a weak signal for the sea urchin output. These weak signals may result in a false positive result for sea urchin. In reality, textures and outlines would not be represented by single nodes, but rather by associated weight patterns of multiple nodes.



Learning occurs by changing connection weights after each piece of data is processed, based on the amount of error in the output compared to the expected result. This is an example of supervised learning, and is carried out through backpropagation.

We can represent the degree of error in an output node j in the n-th data point (training example) by $e_j(n) = d_j(n) - y_j(n)$, where $d_j(n)$ is the desired target value for n-th data point at node j, and $y_j(n)$ is the value produced at node j when the n-th data point is given as an input.

The node weights can then be adjusted based on corrections that minimize the error in the entire output for the n-th data point, given by

$$\mathcal{E}(n) = \frac{1}{2} \sum_{\text{output node } j} e_j^2(n).$$

Using gradient descent, the change in each weight w_{ij} is

$$\Delta w_{ji}(n) = -\eta \frac{\partial \mathcal{E}(n)}{\partial v_j(n)} y_i(n)$$

where $y_i(n)$ is the output of the previous neuron i, and η is the *learning rate*, which is selected to ensure that the weights quickly converge to a response, without oscillations. In the previous expression, $\frac{\partial \mathcal{E}(n)}{\partial v_j(n)}$ denotes the partial derivative of the error $\mathcal{E}(n)$ according to the weighted sum $v_j(n)$ of the input connections of neuron i.

The derivative to be calculated depends on the induced local field v_j, which itself varies. It is easy to prove that for an output node this derivative can be simplified to

$$-\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = e_j(n)\phi'(v_j(n))$$

where ϕ' is the derivative of the activation function described above, which itself does not vary. The analysis is more difficult for the change in weights to a hidden node, but it can be shown that the relevant derivative is

$$-\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = \phi'(v_j(n)) \sum_k -\frac{\partial \mathcal{E}(n)}{\partial v_k(n)} w_{kj}(n).$$

This depends on the change in weights of the kth nodes, which represent the output layer. So to change the hidden layer weights, the output layer weights change according to the derivative of the activation function, and so this algorithm represents a backpropagation of the activation function.[10]

History

Timeline

- Circa 1800, Legendre (1805) and Gauss (1795) created the simplest feedforward network which consists of a single weight layer with linear activation functions. It was trained by the least squares method for minimising mean squared error, also known as linear regression.

Legendre and Gauss used it for the prediction of planetary movement from training data.[11][12][13][14][15]

- In 1943, Warren McCulloch and Walter Pitts proposed the binary artificial neuron as a logical model of biological neural networks.[16]
- In 1958, Frank Rosenblatt proposed the multilayered perceptron model, consisting of an input layer, a hidden layer with randomized weights that did not learn, and an output layer with learnable connections.[17] R. D. Joseph (1960)[18] mentions an even earlier perceptron-like device:[13] "Farley and Clark of MIT Lincoln Laboratory actually preceded Rosenblatt in the development of a perceptron-like device." However, "they dropped the subject."
- In 1960, Joseph[18] also discussed multilayer perceptrons with an adaptive hidden layer. Rosenblatt (1962)[19]:section 16 cited and adopted these ideas, also crediting work by H. D. Block and B. W. Knight. Unfortunately, these early efforts did not lead to a working learning algorithm for hidden units, i.e., deep learning.
- In 1965, Alexey Grigorevich Ivakhnenko and Valentin Lapa published Group Method of Data Handling, the first working deep learning algorithm, a method to train arbitrarily deep neural networks.[20][21] It is based on layer by layer training through regression analysis. Superfluous hidden units are pruned using a separate validation set. Since the activation functions of the nodes are Kolmogorov-Gabor polynomials, these were also the first deep networks with multiplicative units or "gates."[13] It was used to train an eight-layer neural net in 1971.
- In 1967, Shun'ichi Amari reported [22] the first multilayered neural network trained by stochastic gradient descent, which was able to classify non-linearily separable pattern classes. Amari's student Saito conducted the computer experiments, using a five-layered feedforward network with two learning layers.[13]
- In 1970, Seppo Linnainmaa published the modern form of backpropagation in his master thesis (1970).[23][24][13] G.M. Ostrovski et al. republished it in 1971.[25][26] Paul Werbos applied backpropagation to neural networks in 1982[7][27] (his 1974 PhD thesis, reprinted in a 1994 book,[28] did not yet describe the algorithm[26]). In 1986, David E. Rumelhart et al. popularised backpropagation but did not cite the original work.[29][8]
- In 2003, interest in backpropagation networks returned due to the successes of deep learning being applied to language modelling by Yoshua Bengio with co-authors.[30]

Linear regression

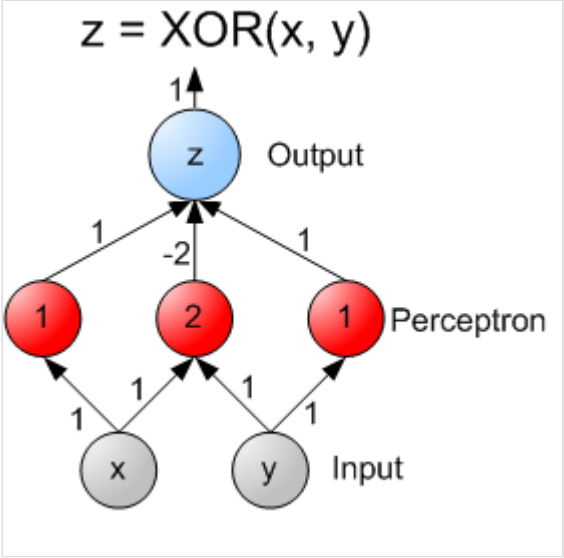
Perceptron

If using a threshold, i.e. a linear activation function, the resulting *linear threshold unit* is called a *perceptron*. (Often the term is used to denote just one of these units.) Multiple parallel non-linear units are able to approximate any continuous function from a compact interval of the real numbers into the interval $[-1,1]$ despite the limited computational power of single unit with a linear threshold function.[31]

Perceptrons can be trained by a simple learning algorithm that is usually called the *delta rule*. It calculates the errors between calculated output and sample output data, and uses this to create an adjustment to the weights, thus implementing a form of gradient descent.

Multilayer perceptron

A **multilayer perceptron** (**MLP**) is a misnomer for a modern feedforward artificial neural network, consisting of fully connected neurons (hence the synonym sometimes used of *fully connected network* (*FCN*)), often with a nonlinear kind of activation function, organized in at least three layers, notable for being able to distinguish data that is not linearly separable.[32]



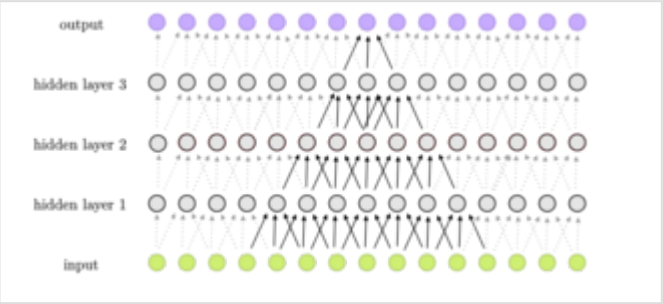
Two-layer neural network capable of calculating XOR. Numbers in neurons represent their explicit threshold. Numbers annotating arrows represent weight of the inputs. If the threshold of 2 is met then a value of 1 is used for the weight multiplication to the next layer. Not meeting the threshold results in 0 being used. The bottom layer of inputs is not always considered a real neural network layer.

Other feedforward networks

Examples of other feedforward networks include convolutional neural networks and radial basis function networks, which use a different activation function.

See also

- Feed forward (control)
- Feedback neural network
- Hopfield network
- Rprop



1D convolutional neural network feed forward example

References

1. Ferrie, C., & Kaiser, S. (2019). *Neural Networks for Babies*. Sourcebooks. ISBN 978-1492671206.
2. Zell, Andreas (1994). *Simulation Neuronaler Netze* [*Simulation of Neural Networks*] (in German) (1st ed.). Addison-Wesley. p. 73. ISBN 3-89319-554-8.
3. Schmidhuber, Jürgen (2015-01-01). "Deep learning in neural networks: An overview". *Neural Networks*. **61**: 85–117. arXiv:1404.7828 (https://arxiv.org/abs/1404.7828). doi:10.1016/j.neunet.2014.09.003 (https://doi.org/10.1016%2Fj.neunet.2014.09.003). ISSN 0893-6080 (https://search.worldcat.org/issn/0893-6080). PMID 25462637 (https://pubmed.ncbi.nlm.nih.gov/25462637). S2CID 11715509 (https://api.semanticscholar.org/CorpusID:11715509).

4. Linnainmaa, Seppo (1970). *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors* (Masters) (in Finnish). University of Helsinki. p. 6–7.
5. Kelley, Henry J. (1960). "Gradient theory of optimal flight paths". *ARS Journal*. **30** (10): 947–954. doi:10.2514/8.5282 (https://doi.org/10.2514%2F8.5282).
6. Rosenblatt, Frank. x. Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms. Spartan Books, Washington DC, 1961
7. Werbos, Paul (1982). "Applications of advances in nonlinear sensitivity analysis" (http://werbos.com/Neural/SensitivityIFIPSeptember1981.pdf) (PDF). *System modeling and optimization*. Springer. pp. 762–770. Archived (https://web.archive.org/web/20160414055503/http://werbos.com/Neural/SensitivityIFIPSeptember1981.pdf) (PDF) from the original on 14 April 2016. Retrieved 2 July 2017.
8. Rumelhart, David E., Geoffrey E. Hinton, and R. J. Williams. "Learning Internal Representations by Error Propagation (https://apps.dtic.mil/dtic/tr/fulltext/u2/a164453.pdf) Archived (https://web.archive.org/web/20221013070443/https://apps.dtic.mil/dtic/tr/fulltext/u2/a164453.pdf) 2022-10-13 at the Wayback Machine". David E. Rumelhart, James L. McClelland, and the PDP research group. (editors), Parallel distributed processing: Explorations in the microstructure of cognition, Volume 1: Foundation. MIT Press, 1986.
9. Achler, T. (2023). "What AI, Neuroscience, and Cognitive Science Can Learn from Each Other: An Embedded Perspective". *Cognitive Computation*.
10. Haykin, Simon (1998). *Neural Networks: A Comprehensive Foundation* (2 ed.). Prentice Hall. ISBN 0-13-273350-1.
11. Merriman, Mansfield. *A List of Writings Relating to the Method of Least Squares: With Historical and Critical Notes*. Vol. 4. Academy, 1877.
12. Stigler, Stephen M. (1981). "Gauss and the Invention of Least Squares" (https://doi.org/10.1214%2Faos%2F1176345451). *Ann. Stat*. **9** (3): 465–474. doi:10.1214/aos/1176345451 (https://doi.org/10.1214%2Faos%2F1176345451).
13. Schmidhuber, Jürgen (2022). "Annotated History of Modern AI and Deep Learning". arXiv:2212.11279 (https://arxiv.org/abs/2212.11279) [cs.NE (https://arxiv.org/archive/cs.NE)].
14. Bretscher, Otto (1995). *Linear Algebra With Applications* (3rd ed.). Upper Saddle River, NJ: Prentice Hall.
15. Stigler, Stephen M. (1986). *The History of Statistics: The Measurement of Uncertainty before 1900* (https://archive.org/details/historyofstatist00stig). Cambridge: Harvard. ISBN 0-674-40340-1.
16. McCulloch, Warren S.; Pitts, Walter (1943-12-01). "A logical calculus of the ideas immanent in nervous activity" (https://doi.org/10.1007/BF02478259). *The Bulletin of Mathematical Biophysics*. **5** (4): 115–133. doi:10.1007/BF02478259 (https://doi.org/10.1007%2FBF02478259). ISSN 1522-9602 (https://search.worldcat.org/issn/1522-9602).
17. Rosenblatt, Frank (1958). "The Perceptron: A Probabilistic Model For Information Storage And Organization in the Brain". *Psychological Review*. **65** (6): 386–408. CiteSeerX 10.1.1.588.3775 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.588.3775). doi:10.1037/h0042519 (https://doi.org/10.1037%2Fh0042519). PMID 13602029 (https://pubmed.ncbi.nlm.nih.gov/13602029). S2CID 12781225 (https://api.semanticscholar.org/CorpusID:12781225).
18. Joseph, R. D. (1960). *Contributions to Perceptron Theory, Cornell Aeronautical Laboratory Report No. VG-11 96--G-7, Buffalo*.
19. Rosenblatt, Frank (1962). *Principles of Neurodynamics*. Spartan, New York.
20. Ivakhnenko, A. G. (1973). *Cybernetic Predicting Devices* (https://books.google.com/books?id=FhwVNQAACAAJ). CCM Information Corporation.

21. Ivakhnenko, A. G.; Grigor′evich Lapa, Valentin (1967). *Cybernetics and forecasting techniques* (https://books.google.com/books?id=rGFgAAAAMAAJ). American Elsevier Pub. Co.
22. Amari, Shun'ichi (1967). "A theory of adaptive pattern classifier". *IEEE Transactions*. **EC** (16): 279-307.
23. Linnainmaa, Seppo (1970). *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors* (Masters) (in Finnish). University of Helsinki. p. 6–7.
24. Linnainmaa, Seppo (1976). "Taylor expansion of the accumulated rounding error". *BIT Numerical Mathematics*. **16** (2): 146–160. doi:10.1007/bf01931367 (https://doi.org/10.1007%2Fbf01931367). S2CID 122357351 (https://api.semanticscholar.org/CorpusID:122357351).
25. Ostrovski, G.M., Volin,Y.M., and Boris, W.W. (1971). On the computation of derivatives. Wiss. Z. Tech. Hochschule for Chemistry, 13:382–384.
26. Schmidhuber, Juergen (25 Oct 2014). "Who Invented Backpropagation?" (https://people.idsia.ch/~juergen/who-invented-backpropagation.html). IDSIA, Switzerland. Archived (https://web.archive.org/web/20240730110408/https://people.idsia.ch/~juergen/who-invented-backpropagation.html) from the original on 30 July 2024. Retrieved 14 Sep 2024.
27. Anderson, James A.; Rosenfeld, Edward, eds. (2000). *Talking Nets: An Oral History of Neural Networks* (https://direct.mit.edu/books/book/4886/Talking-NetsAn-Oral-History-of-Neural-Networks). The MIT Press. doi:10.7551/mitpress/6626.003.0016 (https://doi.org/10.7551%2Fmitpress%2F6626.003.0016). ISBN 978-0-262-26715-1.
28. Werbos, Paul J. (1994). *The Roots of Backpropagation : From Ordered Derivatives to Neural Networks and Political Forecasting*. New York: John Wiley & Sons. ISBN 0-471-59897-6.
29. Rumelhart, David E.; Hinton, Geoffrey E.; Williams, Ronald J. (October 1986). "Learning representations by back-propagating errors" (https://www.nature.com/articles/323533a0). *Nature*. **323** (6088): 533–536. Bibcode:1986Natur.323..533R (https://ui.adsabs.harvard.edu/abs/1986Natur.323..533R). doi:10.1038/323533a0 (https://doi.org/10.1038%2F323533a0). ISSN 1476-4687 (https://search.worldcat.org/issn/1476-4687).
30. Bengio, Yoshua; Ducharme, Réjean; Vincent, Pascal; Janvin, Christian (March 2003). "A neural probabilistic language model" (https://dl.acm.org/doi/10.5555/944919.944966). *The Journal of Machine Learning Research*. **3**: 1137–1155.
31. Auer, Peter; Harald Burgsteiner; Wolfgang Maass (2008). "A learning rule for very simple universal approximators consisting of a single layer of perceptrons" (https://web.archive.org/web/20110706095227/http://www.igi.tugraz.at/harry/psfiles/biopdelta-07.pdf) (PDF). *Neural Networks*. **21** (5): 786–795. doi:10.1016/j.neunet.2007.12.036 (https://doi.org/10.1016%2Fj.neunet.2007.12.036). PMID 18249524 (https://pubmed.ncbi.nlm.nih.gov/18249524). Archived from the original (http://www.igi.tugraz.at/harry/psfiles/biopdelta-07.pdf) (PDF) on 2011-07-06. Retrieved 2009-09-08.
32. Cybenko, G. 1989. Approximation by superpositions of a sigmoidal function *Mathematics of Control, Signals, and Systems*, 2(4), 303–314.

External links

- Feedforward neural networks tutorial (https://web.archive.org/web/20090507210502/http://www.emilstefanov.net/Projects/NeuralNetworks.aspx)
- Feedforward Neural Network: Example (https://web.archive.org/web/20090923121811/http://wiki.syncleus.com/index.php/DANN%3ABackprop_Feedforward_Neural_Network)

- Feedforward Neural Networks: An Introduction (http://media.wiley.com/product_data/excerpt/19/04713491/0471349119.pdf)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Feedforward_neural_network&oldid=1326004759"